

הנדסת מערכות חלונאיות

עקרונות, תבניות וכלים מבוססי פייתון ו-C#



יוסי זגורי

Yossi.Zaguri@gmail.com

052-4668866



Qt/PySide Introduction

Qt and PySide6

- ❑ Qt is An application development open-source *framework* for cross-platform GUI development, includes widget toolkit, multimedia support, working with DB and vector graphics, Written in C++.
- ❑ Event-driven (as opposed to REPL).
- ❑ Signals & Slots : Events happen to Widgets, emitting a *Signal (message)* being handled by *Slots*.
- ❑ Two versions of usage in Python: PyQt & PySide - the same in general and differ only on commercial aspects.
- ❑ Support of native platform API's (Windows, iOS, Linux & Android).
- ❑ Supports the use of [QtQuick](#) & a declarative scripting language(QML) to define user interface.
- ❑ As PySide6 is Python binding (wrapper) for Qt, coding standard common in Python (`more_stuff`) will not be adhered , rather appear as `moreStuff` C++ Camel case convention.

Qt and PySide6

- ☐ Adding Widgets
- ☐ Use layouts
- ☐ Connect Widgets to slots

```
pip install pyside6 (windows)  
pip3 install pyside6(macOS and Linux)
```

Qt Creator (not free) :<https://www.qt.io/download>

First App

- ☐ Adding Widgets
- ☐ Use layouts
- ☐ Connect Widgets to slots

```
pip install pyside6 (windows)  
pip3 install pyside6(macOS and Linux)
```

Qt Creator (not free) :<https://www.qt.io/download>

Tutorial :<https://www.pythonguis.com/pyside6/>

First App

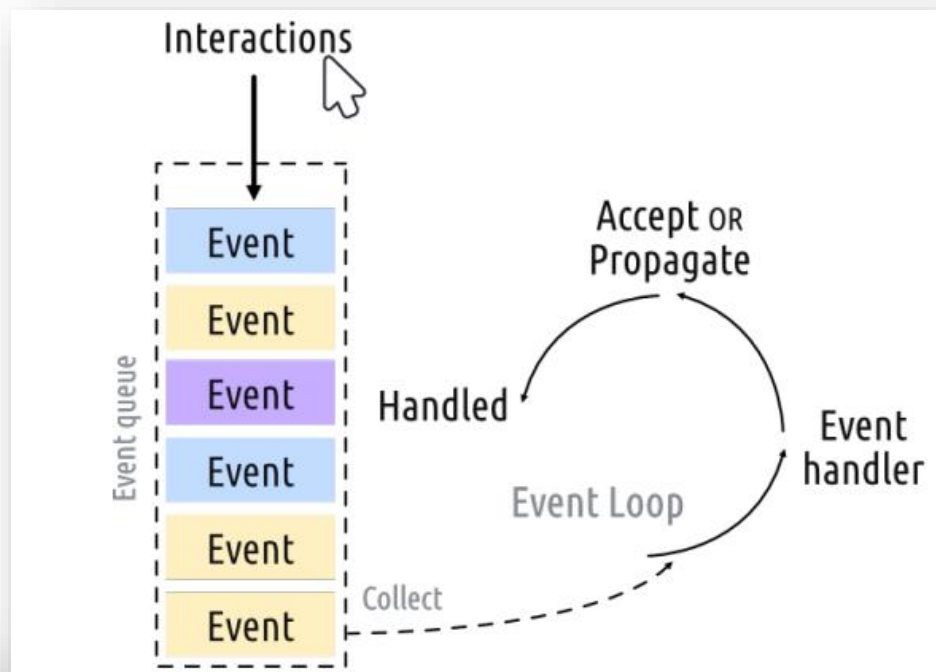
```
from PySide6.QtWidgets import QApplication, QWidget

app = QApplication([])
window = QWidget()
window.show()
app.exec()
```

- ☐ QApplication class is The core of every Qt Application. Every application needs one and only one (Application root).
- ☐ Another main modules for Qt is *QtWidgets*.
- ☐ In Qt all top-level widgets are windows.
- ☐

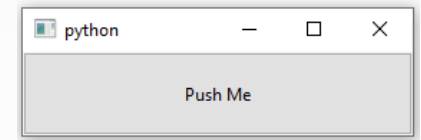
Qapplication Event Loop

- ❑ QApplication holds the **event loop** of an application - core loop which governs all user interaction with the GUI
- ❑ User Interaction generates event placed on the event queue
- ❑ Queue is checked on each iteration and if a waiting event is found, the event and control is passed to the specific event handler, after that the control goes back to the event loop
- ❑ Each application has only one running event loop

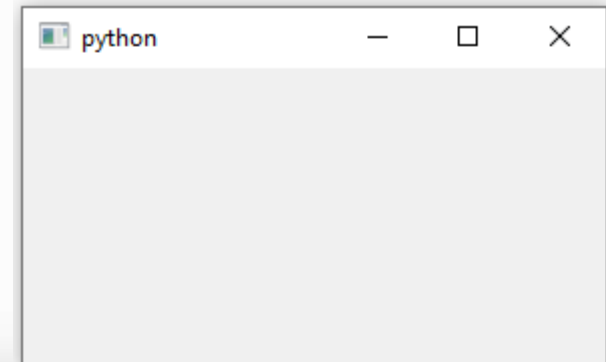


QMainWindow

```
from PySide6.QtWidgets import QApplication, QPushButton
app = QApplication([])
window = QPushButton("Push Me")
window.show()
app.exec()
```



- ❑ Any widget is *a window*, yet it is not useful unless nesting using layout is applied
- ❑ QMainWindow is Ready made widget supporting toolbars, menus, statusbar, dockage widgets etc.
- ❑ Createing custom window is done by subclassing QMainWindow



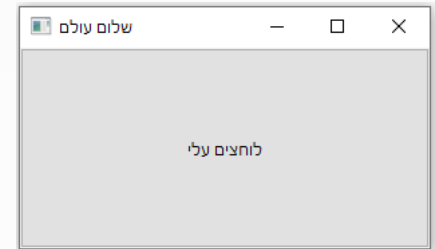
QMainWindow

```
from PySide6.QtCore import QSize, Qt
from PySide6.QtWidgets import (
    QApplication,
    QMainWindow,
    QPushButton,
)

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("שלום עולם")
        button = QPushButton("לוחצים עלי")
        self.setCentralWidget(button)

def main():
    app = QApplication([])
    window = MainWindow()
    window.show()
    app.exec()

if __name__=="__main__":
    main()
```



`setCentralWidget()` is a QMainWindow specific function allowing to set the widget that goes in the middle of the window

[Widgets Classes | Qt Widgets 5.15.16](#)

Window Sizing



```
class MainWindow(QMainWindow):  
    def __init__(self):  
        super().__init__()  
        self.setWindowTitle("My App")  
        button = QPushButton("Press Me!")  
        self.setFixedSize(QSize(400, 300))
```

☐ Sizing Methods:

- ☐ .setFixedSize()
- ☐ .setMinimumSize()
- ☐ .setMaximumSize()

☐ Relevant to any widget

Window Sizing

```
class MainWindow(QMainWindow):  
    def __init__(self):  
        super().__init__()  
        self.setWindowTitle("My App")  
        button = QPushButton("Press Me!")  
        self.setFixedSize(QSize(400, 300))
```

☐ Sizing Methods:

- ☐ .setFixedSize()
- ☐ .setMinimumSize()
- ☐ .setMaximumSize()

☐ Relevant to any widget

Widgets

- ❑ UI Component that the user can interact with (Controls in other frameworks)
- ❑ User interfaces are made up of multiple widgets, arranged within the window, placed with a layout manager.
- ❑ Qt comes with a large selection of widgets available , and even allows you to create your own custom widgets.

```
from PySide6.QtCore import Qt
from PySide6.QtWidgets import QVBoxLayout # <2>
from PySide6.QtWidgets import QWidget # <1>
from PySide6.QtWidgets import (
    QApplication,
    QCheckBox,
    QComboBox,
    QDateEdit,
    QDateTimeEdit,
    QDial,
    QDoubleSpinBox,
    QFontComboBox,
    QLabel,
    QLCDNumber,
    QLineEdit,
    QMainWindow,
    QProgressBar,
    QPushButton,
    QRadioButton,
    QSlider, QSpinBox, QTimeEdit,
)
```

```
def main():
    app = QApplication([])
    window = MainWindow()
    window.show()
    app.exec()
```

Widgets



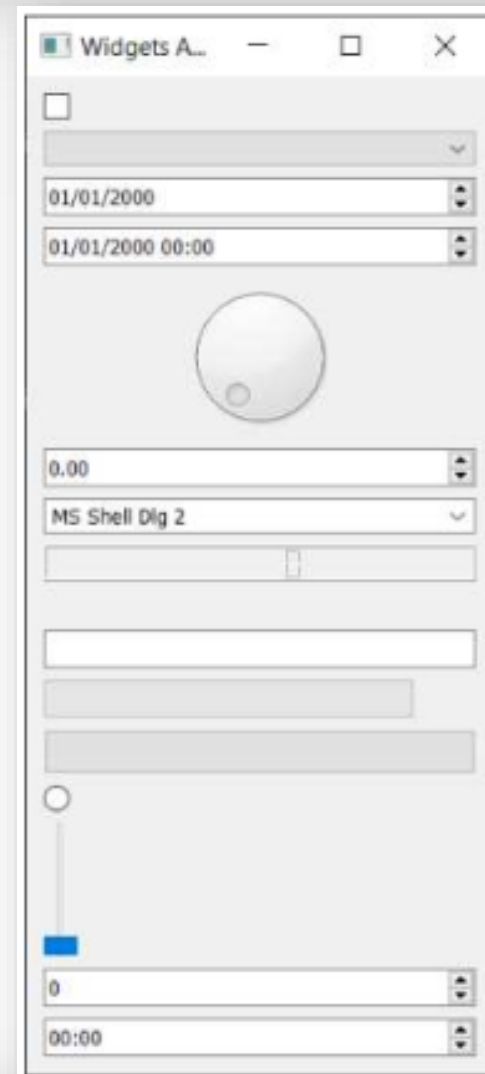
```
class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()

        self.setWindowTitle("Widgets App")

        layout = QVBoxLayout()
        widgets = [
            QCheckBox,
            QComboBox,
            QDateEdit,
            QDateTimeEdit,
            QDial,
            QDoubleSpinBox,
            QFontComboBox,
            QLCDNumber,
            QLabel,
            QLineEdit,
            QProgressBar,
            QPushButton,
            QRadioButton,
            QSlider, QSpinBox, QTimeEdit,
        ]

        for w in widgets:
            layout.addWidget(w())

        widget = QWidget()
        widget.setLayout(layout)
        # to take up all the space in the window by default.
        self.setCentralWidget(widget)
```



Basic Widget List



Widget	What it does
<code>QCheckBox</code>	A checkbox
<code>QComboBox</code>	A dropdown list box
<code>QDateEdit</code>	For editing dates
<code>QDateTimeEdit</code>	For editing dates and datetimes
<code>QDial</code>	Rotatable dial
<code>QDoubleSpinBox</code>	A number spinner for floats
<code>QFontComboBox</code>	A list of fonts
<code>QLCDNumber</code>	A quite ugly LCD display
<code>QLabel</code>	Just a label, not interactive
<code>QLineEdit</code>	Enter a line of text
<code>QProgressBar</code>	A progress bar
<code>QPushButton</code>	A button
<code>QRadioButton</code>	A group with only one active
<code>QSlider</code>	A slider
<code>QSpinBox</code>	An integer spinner
<code>QTimeEdit</code>	For editing times

```
widget = QLabel("Hello")
font = widget.font()
font.setPointSize(30)
widget.setFont(font)
widget.setAlignment(Qt.AlignHCenter | Qt.AlignVCenter)
```

Horizontal alignment

Flag	Behavior
<code>Qt.AlignLeft</code>	Aligns with the left edge.
<code>Qt.AlignRight</code>	Aligns with the right edge.
<code>Qt.AlignHCenter</code>	Centers horizontally in the available space.
<code>Qt.AlignJustify</code>	Justifies the text in the available space.

Vertical alignment

Flag	Behavior
<code>Qt.AlignTop</code>	Aligns with the top.
<code>Qt.AlignBottom</code>	Aligns with the bottom.
<code>Qt.AlignVCenter</code>	Centers vertically in the available space.

QLabel holding Image

```
from PySide6.QtGui import QPixmap
from PySide6.QtWidgets import QApplication,
QLabel, QMainWindow
```

```
class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Display Image")
        widget = QLabel("אנחנו ננצח")
        widget.setPixmap(QPixmap("www.jpg"))
        self.setCentralWidget(widget)
```

```
def main():
    app = QApplication([])
    window = MainWindow()
    window.show()
    app.exec()
```

```
if __name__ == "__main__":
    main()
```

Addressing Relative Path:

```
import os
import sys
```

```
basedir = os.path.dirname(__file__)
print("Current working folder:", os.getcwd())
print("Paths are relative to:", basedir)
```

```
...
widget = QLabel("אנחנו ננצח")
widget.setPixmap(
    QPixmap(os.path.join(basedir, "www.jpg")))
```

Stretch and scale to window size:

```
widget.setScaledContents(True)
```

QCheckBox



```
from PySide6.QtCore import Qt
from PySide6.QtWidgets import QApplication, QCheckBox,
QMainWindow
class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("My App")
        widget = QCheckBox("This is a checkbox")
        widget.setCheckState(Qt.Checked)
        widget.stateChanged.connect(self.show_state)
        self.setCentralWidget(widget)
    def show_state(self, s):
        print(s == Qt.Checked)
        print(s)

#create main
app = QApplication([])
window = MainWindow()
window.show()
app.exec()
```


QCheckBox

```
from PySide6.QtCore import Qt
from PySide6.QtWidgets import QApplication, QComboBox, QMainWindow

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("My App")
        widget = QComboBox()
        widget.addItems(["One", "Two", "Three"])
        widget.currentIndexChanged.connect(self.index_changed)
        widget.currentTextChanged.connect(self.text_changed)
        self.setCentralWidget(widget)
    def index_changed(self, i): # i is an int
        print(i)
    def text_changed(self, s): # s is a str
        print(s)

#create main
app = QApplication([])
window = MainWindow()
window.show()
app.exec()
```

Add to enable edit

`widget.setEditable(True)`

QLineEdit

```
from PySide6.QtCore import Qt
from PySide6.QtWidgets import QApplication, QLineEdit, QMainWindow

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("My App")
        widget = QLineEdit()
        widget.setMaxLength(10)
        widget.setPlaceholderText("Enter your text")
        # widget.setReadOnly(True) # uncomment this to make readonly
        widget.returnPressed.connect(self.return_pressed)
        widget.selectionChanged.connect(self.selection_changed)
        widget.textChanged.connect(self.text_changed)
        widget.textEdited.connect(self.text_edited)
        self.setCentralWidget(widget)
    def return_pressed(self):
        print("Return pressed!")
        self.centralWidget().setText("BOOM!")
    def selection_changed(self):
        print("Selection changed")
        print(self.centralWidget().selectedText())
    def text_changed(self, s):
        print("Text changed...")
        print(s)
    def text_edited(self, s):
        print("Text edited...")
        print(s)

app = QApplication([])
window = MainWindow()
window.show()
app.exec()
```

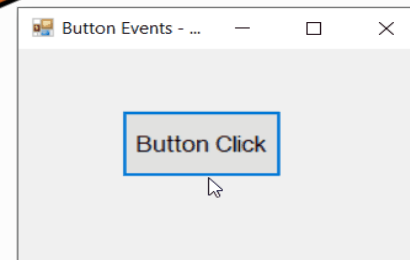
Add Input Mask

```
widget.setInputMask('000.000.000.000;_')
```

QWidget is used to create an empty window in the examples. But QWidget can also be used as a *container* for other widgets, together with Layouts to construct windows or compound widgets.

Qt events & signals

- ❑ Signals and slots are a higher-level mechanism for communication between objects, often used for widget interactions and custom signals within your classes.
- ❑ Events are lower-level and typically used for handling input and other system-level events.



Signals & Slots

- ❑ Signals are notifications emitted by widgets when something happens
- ❑ *Slots* is the name Qt uses for the receivers of signals

```
from PySide6.QtWidgets import (
    QApplication,
    QMainWindow,
    QPushButton,
)

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("My App")
        button = QPushButton("Press Me!")
        button.setCheckable(True)
        button.clicked.connect(self.button_clicked)
        self.setCentralWidget(button)

    def button_clicked(self):
        print("Clicked!")

def main():
    app = QApplication([])
    window = MainWindow()
    window.show()
    app.exec()

main()
```

Signals & Slots

```
from PySide6.QtWidgets import (
    QApplication,
    QMainWindow,
    QPushButton,
)

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("My App")
        button = QPushButton("Press Me!")
        button.setCheckable(True)

        button.clicked.connect(self.the_button_was_clicked)
        button.clicked.connect(self.the_button_was_toggled)

        self.setCentralWidget(button)

    def the_button_was_clicked(self):
        print("Clicked!")

    def the_button_was_toggled(self, checked):
        print("Checked?", checked)

app = QApplication([])
window = MainWindow()
window.show()

app.exec()
```

- ❑ Signals are notifications e

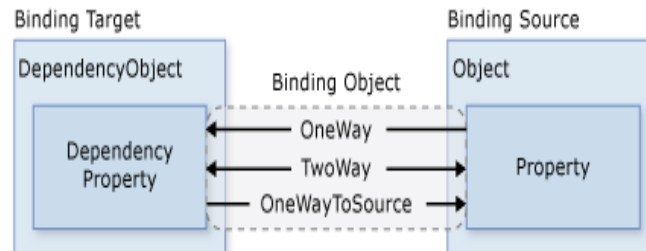
Signals & Slots

- ❑ Sometimes container member needed to handle state (if one is not supplied by the widget itself)
- ❑ To be define ... : WHERE EXACTLY ?

```
...  
self.button.released.connect(self.the_button_was_released)  
  
...  
def the_button_was_released(self):  
    self.button_is_checked = self.button.isChecked()
```

- ❑ Setting Widget Properties:
- ❑ It seems as “step back” from WPF ... Any suggestions to fix it ?

```
self.button.setText("You already clicked me."  
)  
#change title  
self.setWindowTitle("A new window title")  
# slot for title change  
self.windowTitleChanged.connect(handler)
```



“Binding”



```
from PySide6.QtWidgets import (
    QApplication,
    QMainWindow,
    QLabel,
    QLineEdit,
    QVBoxLayout,
    QWidget,
)

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("My App")
        self.label = QLabel()
        self.input = QLineEdit()
        self.input.textChanged.connect(self.label.setText)
        layout = QVBoxLayout()
        layout.addWidget(self.input)
        layout.addWidget(self.label)
        container = QWidget()
        container.setLayout(layout)
        # Set the central widget of the Window.
        self.setCentralWidget(container)

def main():
    app = QApplication([])
    window = MainWindow()
    window.show()
    app.exec()

main()
```

- ❑ Notice the Connection of two widgets together
- ❑ When widget exposes built in slots they are called “Public Slots”
- ❑ QVBoxLayout : Applying layout manager.

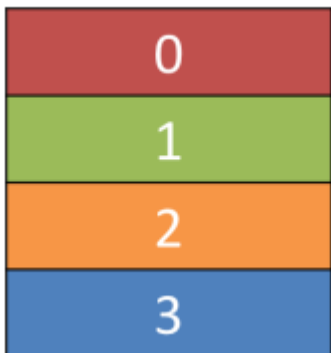
Layouts

Layout	Behavior
QHBoxLayout	Linear horizontal layout
QVBoxLayout	Linear vertical layout
QGridLayout	In indexable grid XxY
QStackedLayout	Stacked (z) in front of one another

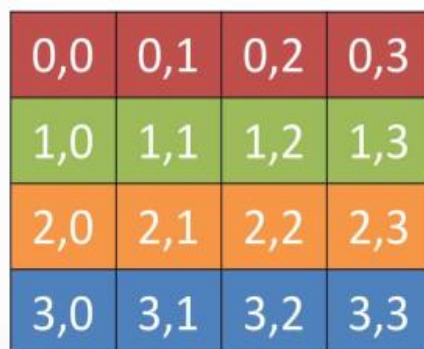
QHBoxLayout



QVBoxLayout



QGridLayout



- ❑ Layouts can be nested
- ❑ Setting margins & spacing

```
layout1.setContentsMargins(0,0,0,0)  
layout1.setSpacing(20)
```

```
layout = QStackedLayout()  
layout.setCurrentIndex(3)
```

QStackedLayout

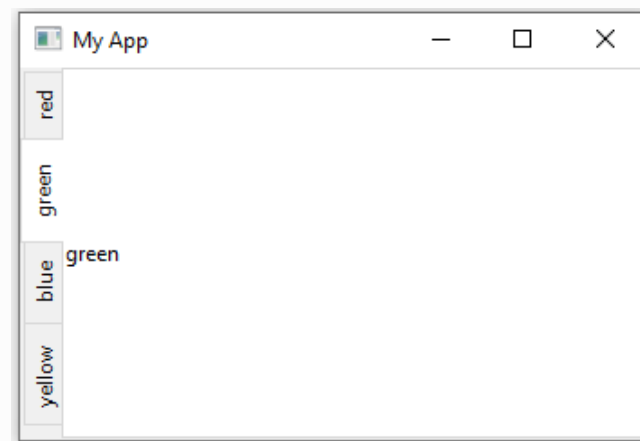


Tabs

```
from PySide6.QtWidgets import (
    QApplication,
    QLabel,
    QMainWindow,
    QPushButton,
    QTabWidget,
    QWidget,
)

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("My App")
        tabs = QTabWidget()
        tabs.setTabPosition(QTabWidget.West)
        tabs.setMovable(True)
        for n, color in enumerate(["red", "green", "blue", "yellow"]):
            tabs.addTab(QLabel(color), color)
        self.setCentralWidget(tabs)

app = QApplication([])
window = MainWindow()
window.show()
app.exec()
```



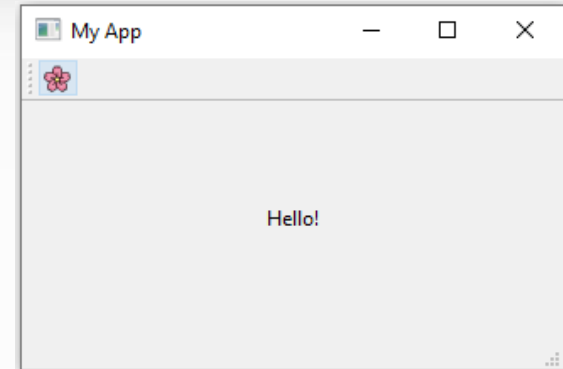
Toolbars



```
import os
import sys
from PySide6.QtCore import QSize, Qt
from PySide6.QtGui import QAction, QIcon
from PySide6.QtWidgets import (
    QApplication,
    QLabel,
    QMainWindow,
    QStatusBar,
    QToolBar,
)
basedir = os.path.dirname(__file__)

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("My App")
        label = QLabel("Hello!")
        label.setAlignment(Qt.AlignCenter)
        self.setCentralWidget(label)
        toolbar = QToolBar("My main toolbar")
        toolbar.setIconSize(QSize(16, 16))
        self.addToolBar(toolbar)
        button_action = QAction(QIcon(os.path.join(basedir, "flower.png")), "Your button", self)
        button_action.setStatusTip("This is your button")
        button_action.triggered.connect(self.onMyToolBarButtonClick)
        button_action.setCheckable(True)
        toolbar.addAction(button_action)
        self.setStatusBar(QStatusBar(self))

    def onMyToolBarButtonClick(self, s):
        print("click", s)
```



```
app = QApplication(sys.argv)
window = MainWindow()
window.show()
app.exec()
```

Menus & Dialogs



```
import sys
from PySide6.QtWidgets import QApplication, QMainWindow, QMenu, QMenuBar, QFileDialog

class MainWindow(QMainWindow):
    def __init__(self):
        super(MainWindow, self).__init__()
        self.init_ui()

    def init_ui(self):
        menubar = self.menuBar()
        file_menu = menubar.addMenu('File')
        file_menu.addAction('Open', self.open_file)
        file_menu.addSeparator()
        file_menu.addAction('Exit', self.close)
        self.setGeometry(100, 100, 800, 600)
        self.setWindowTitle('PySide6 Menu Example')

    def open_file(self):
        options = QFileDialog.Options()
        options |= QFileDialog.ReadOnly

        file_dialog = QFileDialog()
        file_dialog.setOptions(options)
        file_dialog.fileSelected.connect(self.load_file)

        if file_dialog.exec_() == QFileDialog.Accepted:
            pass # The fileSelected signal will be handled in load_file method

    def load_file(self, file_path):
        print(f'Opening file: {file_path}')
        # Implement your file loading logic here

if __name__ == '__main__':
    app = QApplication([])
    main_window = MainWindow()
    main_window.show()
    app.exec()
```

Dialog Buttons

QDialogButtonBox available button types

Button types

`QDialogButtonBox.Ok`

`QDialogButtonBox.Open`

`QDialogButtonBox.Save`

`QDialogButtonBox.Cancel`

`QDialogButtonBox.Close`

`QDialogButtonBox.Discard`

`QDialogButtonBox.Apply`

`QDialogButtonBox.Reset`

`QDialogButtonBox.RestoreDefaults`

`QDialogButtonBox.Help`

`QDialogButtonBox.SaveAll`

`QDialogButtonBox.Yes`

`QDialogButtonBox.YesToAll`

`QDialogButtonBox.No`

`QDialogButtonBox.NoToAll`

`QDialogButtonBox.Abort`

`QDialogButtonBox.Retry`

`QDialogButtonBox.Ignore`

`QDialogButtonBox.NoButton`

```
buttons = QDialogButtonBox.Ok | QDialogButtonBox.Cancel
self.buttonBox = QDialogButtonBox(buttons)
self.buttonBox.accepted.connect(self.accept)
self.buttonBox.rejected.connect(self.reject)
```

Message Boxes

```
QMessageBox.about(parent, title, message)
QMessageBox.critical(parent, title, message)
QMessageBox.information(parent, title, message)
QMessageBox.question(parent, title, message)
QMessageBox.warning(parent, title, message)
```

```
def button_clicked(self, s):
    button = QMessageBox.question(self, "Question dialog",
    "message")
    if button == QMessageBox.Yes:
        print("Yes!")
    else:
        print("No!")
```

Dialog Open file

QDialogButtonBox available button types

Button types

<code>QDialogButtonBox.Ok</code>
<code>QDialogButtonBox.Open</code>
<code>QDialogButtonBox.Save</code>
<code>QDialogButtonBox.Cancel</code>
<code>QDialogButtonBox.Close</code>
<code>QDialogButtonBox.Discard</code>
<code>QDialogButtonBox.Apply</code>
<code>QDialogButtonBox.Reset</code>
<code>QDialogButtonBox.RestoreDefaults</code>
<code>QDialogButtonBox.Help</code>
<code>QDialogButtonBox.SaveAll</code>
<code>QDialogButtonBox.Yes</code>
<code>QDialogButtonBox.YesToAll</code>
<code>QDialogButtonBox.No</code>
<code>QDialogButtonBox.NoToAll</code>
<code>QDialogButtonBox.Abort</code>
<code>QDialogButtonBox.Retry</code>
<code>QDialogButtonBox.Ignore</code>
<code>QDialogButtonBox.NoButton</code>

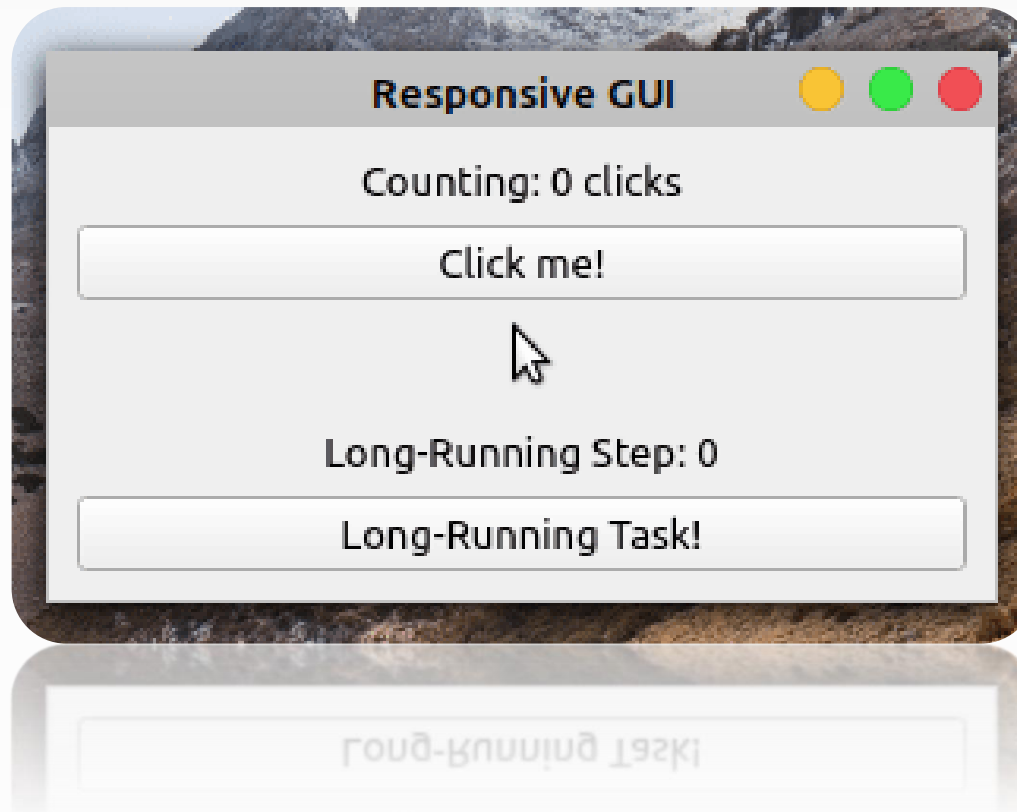
```
buttons = QDialogButtonBox.Ok | QDialogButtonBox.Cancel
self.buttonBox = QDialogButtonBox(buttons)
self.buttonBox.accepted.connect(self.accept)
self.buttonBox.rejected.connect(self.reject)
```

Message Boxs

```
QMessageBox.about(parent, title, message)
QMessageBox.critical(parent, title, message)
QMessageBox.information(parent, title, message)
QMessageBox.question(parent, title, message)
QMessageBox.warning(parent, title, message)
```

```
def button_clicked(self, s):
    button = QMessageBox.question(self, "Question dialog",
    "message")
    if button == QMessageBox.Yes:
        print("Yes!")
    else:
        print("No!")
```

Threads & Responsive GUI



<https://realpython.com/python-pyqt-qthread/>

Calculator Demo

calculator.py

```
from PySide6.QtWidgets import QMainWindow,
QWidget, QVBoxLayout, QPushButton, QGridLayout, QLineEdit

class CalculatorApp(QMainWindow):
    def __init__(self):
        super().__init__()

        self.setWindowTitle("0.0.Calculator")
        self.setGeometry(100, 100, 400, 200)

        self.central_widget = QWidget(self)
        self.setCentralWidget(self.central_widget)

        self.layout = QVBoxLayout(self.central_widget)

        self.result_display = QLineEdit(self)
        self.layout.addWidget(self.result_display)

        buttons = [
            '7', '8', '9', '/',
            '4', '5', '6', '*',
            '1', '2', '3', '-',
            '0', '.', '=', '+' ]

        self.create_buttons(buttons)
```

main.py

```
import sys
from PySide6.QtWidgets import
QApplication
from calculator import *

def main():
    app = QApplication([])
    calculator = CalculatorApp()
    calculator.show()
    app.exec()

if __name__ == "__main__":
    main()
```

Calculator Demo



```
def create_buttons(self, buttons):
    grid_layout = QGridLayout()
    self.layout.addLayout(grid_layout)

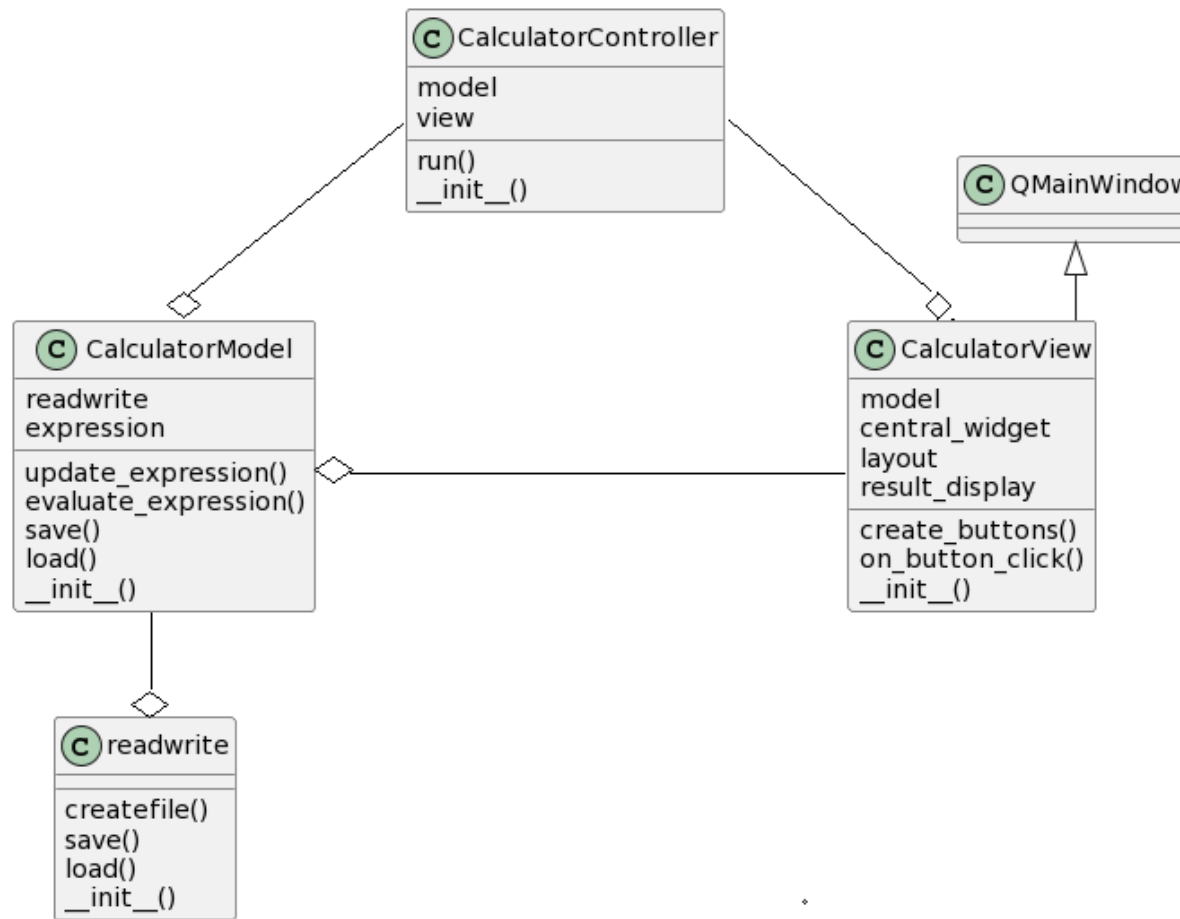
    row = 0
    col = 0

    for button_text in buttons:
        button = QPushButton(button_text, self)
        button.clicked.connect(self.on_button_click)
        grid_layout.addWidget(button, row, col)
        col += 1
        if col > 3:
            col = 0
            row += 1

    def on_button_click(self):
        button = self.sender() # get a pointer to the pressed button
        current_text = self.result_display.text()

        if button.text() == '=':
            try:
                result = eval(current_text)
                self.result_display.setText(str(result))
            except Exception as e:
                self.result_display.setText("Error")
        else:
            new_text = current_text + button.text()
            self.result_display.setText(new_text)
```


MVC Calculator



<https://realpython.com/python-pyqt-gui-calculator/>

MVC Calculator



main.py

```
import sys
from PySide6.QtWidgets import QApplication

from model import CalculatorModel
from view import CalculatorView
from controller import CalculatorController

def main():
    app = QApplication()
    model = CalculatorModel()
    view = CalculatorView(model)
    controller = CalculatorController(model, view)
    controller.run()
    app.exec()

if __name__ == "__main__":
    main()
```

MVC Calculator



controller.py

```
# מנתב את הקוד בהתאם לצורך ולמצב
class CalculatorController:
    def __init__(self, model, view):
        self.model = model
        self.view = view
    def run(self):
        self.view.show()
```

MVC Calculator



view.py

```
from PySide6.QtWidgets import QApplication, QMainWindow, QWidget, QVBoxLayout, QPushButton, QLineEdit,
QGridLayout

# אחראי על התצוגה והאינטראקציה עם המשתמש
class CalculatorView(QMainWindow):
    def __init__(self, model):
        super().__init__()

        self.model = model

        self.setWindowTitle("Calculator")
        self.setGeometry(100, 100, 400, 200)

        self.central_widget = QWidget(self)
        self.setCentralWidget(self.central_widget)

        self.layout = QVBoxLayout(self.central_widget)

        self.result_display = QLineEdit(self)
        self.layout.addWidget(self.result_display)

        buttons = [
            '7', '8', '9', '/',
            '4', '5', '6', '*',
            '1', '2', '3', '-',
            '0', '.', '=', '+'
        ]

        self.create_buttons(buttons)
```

MVC Calculator



view.py –cont'

```
def create_buttons(self, buttons):
    grid_layout = QGridLayout()
    self.layout.addLayout(grid_layout)

    row = 0
    col = 0

    for button_text in buttons:
        button = QPushButton(button_text, self)
        button.clicked.connect(self.on_button_click)
        grid_layout.addWidget(button, row, col)

        col += 1
        if col > 3:
            col = 0
            row += 1

    #add save
    button = QPushButton("save", self)
    button.clicked.connect(self.on_button_click)
    grid_layout.addWidget(button, 5,1)

    #add show
    button = QPushButton("show", self)
    button.clicked.connect(self.on_button_click)
    grid_layout.addWidget(button, 5,2)
```

MVC Calculator

view.py cont'

```
def on_button_click(self):  
    # איתור הכפתור שהפעיל את האירוע  
    button = self.sender()  
    button_text = button.text()  
    print(button_text)  
  
    if button_text == '=':  
        result = self.model.evaluate_expression()  
        self.result_display.setText(result)  
    elif button_text == "show":  
        self.model.load()  
        self.result_display.setText(str(self.model.expression))  
    elif button_text == "save":  
        self.model.save()  
    else:  
        self.model.update_expression(button_text)  
        self.result_display.setText(self.model.expression)
```

MVC Calculator

model.py

```
from ReaderWriter import readwrite

# זוכר את הנתונים ומבצע חישובים ולוגיקה עבורם
class CalculatorModel:
    def __init__(self):
        self.expression = ""
        self.readwrite=readwrite()

    def update_expression(self, value):
        self.expression += str(value)

    def evaluate_expression(self):
        try:
            result = eval(self.expression)
            self.expression=""
            return str(result)
        except Exception as e:
            return "Error"

    def save(self):
        self.readwrite.save(self.expression)

    def load(self):
        self.expression=self.readwrite.load()
```

MVC Calculator



readerwriter.py

```
import os
import csv

class readwrite():
    def __init__(self):
        self.filename="history.csv"
        self.createfile()

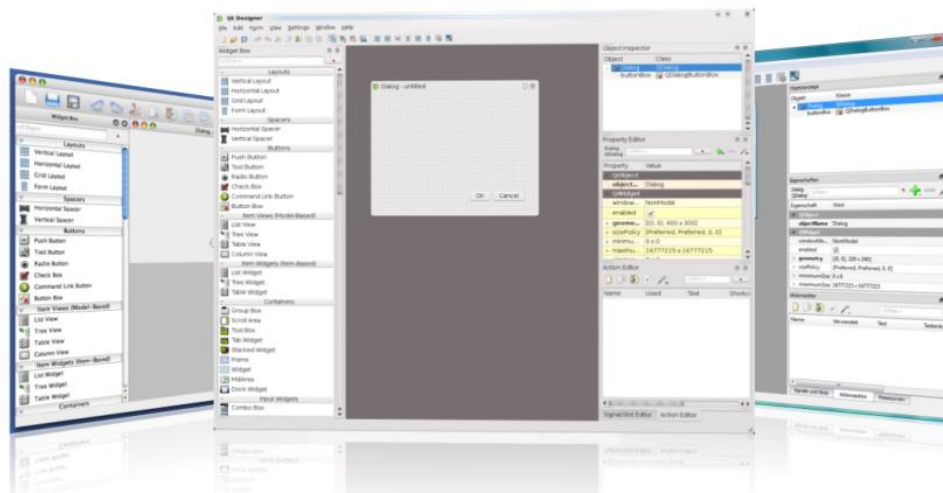
    def createfile(self):
        if not os.path.isfile(self.filename):
            with open(self.filename, 'w', newline='') as csv_file:
                csv_writer = csv.writer(csv_file)
                csv_writer.writerow(["Saved Calculations"])

    def save(self,newrow):
        newrow=[newrow]
        with open(self.filename, 'a', newline='') as csv_file:
            csv_writer = csv.writer(csv_file)
            csv_writer.writerow(newrow)

    def load(self):
        result=None
        with open(self.filename, 'r', newline='') as csv_file:
            csv_reader = csv.reader(csv_file)
            lines = list(csv_reader)
            text=str(lines[-1][0])

            if not (text.__contains__("Saved")):
                return text
```


Qt Designer



Tutorial : <https://doc.qt.io/qtforpython-6/tutorials/basictutorial/uifiles.html>

Compile QML: `pyside6-uic mainwindow.ui -o ui_mainwindow.py`

